



AI-Based Energy Reconstruction for the ATLAS Tile Calorimeter at HL-LHC

From QAT to the FPGA: bit-width study and the hls4ml fixed-point port

Bibhor Puan | GSoC 2026 contributor

Mentors: Luca Fiorini, Fernando Carrio Argos

IFIC (CSIC – Universitat de Valencia) / ATLAS TileCal

Where things stand

- The model carried forward is a 91-parameter 1D-CNN on a 9-sample window, with one dual-gain input $S = \text{HG} + 4096 \times \text{LG}$ and one energy output.
- Every number in this talk is the calibrated per-bin sigma band on the full A1-A5 test set. The band is quoted rather than the global sigma, which the near-zero pile-up bin dominates.
- The reference point is the float student: 127.0 HG ADC, 3.43 LG ADC.
- Two things changed since 6 July. The weight sweep finished and overturned an earlier conclusion, and the model now runs through hls4ml as a bit-accurate simulation of the firmware.

The quantized model

```
def act(bits):    # signed, per-tensor, power-of-2 scale
    return dict(act_quant=Int8ActPerTensorFixedPoint,
                 bit_width=bits, return_quant_tensor=True)

def wgt(bits):
    return dict(weight_quant=Int8WeightPerTensorFixedPoint,
                 weight_bit_width=bits, bias_quant=Int32Bias)

self.quant_inp = qnn.QuantIdentity(**act(input_act))
self.conv1 = qnn.QuantConv1d(1, 4, 5, padding=2, **wgt(w_bits))
self.act1 = nn.LeakyReLU(0.25)           # 2^-2, a bit shift on the FPGA
self.qact1 = qnn.QuantIdentity(**act(inner_act))
self.conv2 = qnn.QuantConv1d(4, 3, 3, padding=1, **wgt(w_bits))
self.act2 = nn.LeakyReLU(0.25)
self.qact2 = qnn.QuantIdentity(**act(inner_act))
self.fc = qnn.QuantLinear(3 * 9, 1, **wgt(fc_bits)) # 24+39+28 = 91
```

From train/models.py, with the config variables replaced by their values. The scales are power-of-2 and per-tensor, the case hls4ml documents as safe. No BatchNorm, no dilation, no residual.

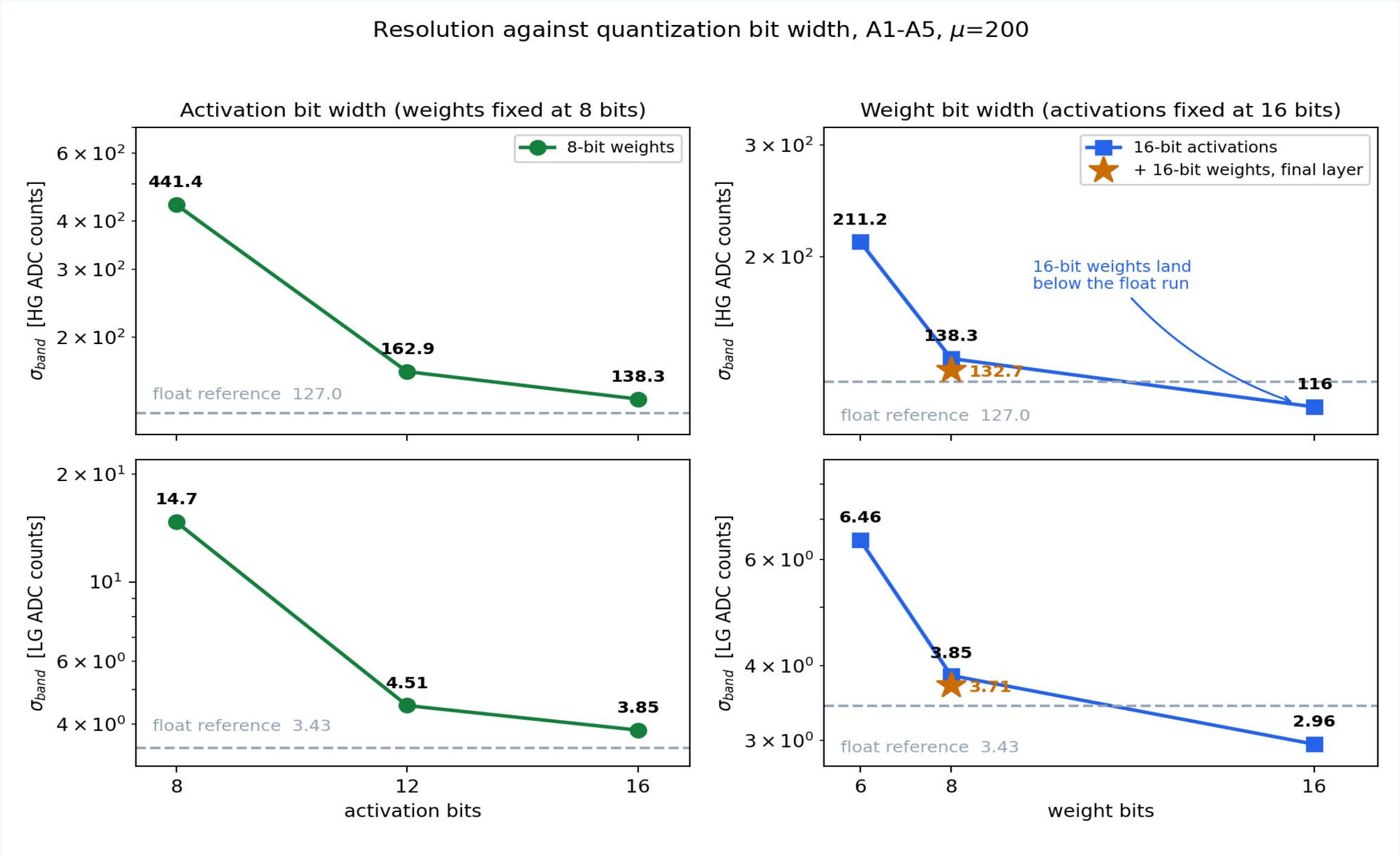
Resolution against bit width

Calibrated per-bin sigma band, HG ADC and LG ADC, full A1-A5 test set

Configuration	HG band	LG band	Comment
w8 / a8	441.4	14.7	unusable
w8 / a12	162.9	4.51	lighter fallback
w8 / a16	138.3	3.85	the one ported to firmware
+ fc weights 16b	132.7	3.71	28 weights changed
w6 / a16	211.2	6.46	weights too narrow
w16 / a16	116.0	2.96	the new run
float student	127.0	3.43	the reference

- At low precision the activations dominate, 8 to 12 bits removes most of the damage.
- The weights are not free either. Eight-bit weights cost about 22 HG ADC of band against sixteen.

Resolution against bit width



bits_vs_error.png, four panels. Top row HG band, bottom row LG band. Left column, activation bit width at fixed 8-bit weights. Right column, weight bit width at fixed 16-bit activations. Dashed line is the float reference.

Where 8 bits breaks down

- The output has to span 0–4095 ADC. Eight activation bits give roughly 256 levels, so E_reco comes out in steps of about 16 ADC. That is the banding seen at low energy.
- Conv and LeakyReLU are both piecewise linear, so neither compresses the range. The narrowest quantizer in the chain ends up capping the network.

Attempt at 8 bits	HG band	What went wrong
16b input, 8b inner	340	recrushed at the first inner quantizer
Per-gain 2-channel	424	per-tensor scale, LG rounds to 0
Log-compressed input	diverged	adds no output levels
Log-compressed target	911	multiplicative error, high-E tail

None of the four adds output levels, which is the likely reason none of them helped.

The weights, not the input encoding, were the limit

The three weight-sweep runs share a starting checkpoint, a seed, a learning rate and a schedule, so comparing them should isolate the weight bits.

Run	HG band	best val loss
w6 / a16	211.2	-
w8 / a16	138.3	5.46e-4
w16 / a16	116.0	4.55e-4
float student	127.0	5.51e-4

- An earlier conclusion held that the weights were not the bottleneck. It rested only on 6 bits being worse than 8. The sweep never went upward, and the headroom was upward.
- Validation loss is the plain RMSE+MAE throughout, with no distillation term, so the runs should be comparable.
- w16/a16 also lands below the float run. That was not expected and is not yet explained.
- It stopped at the 100-epoch cap with validation loss still falling, so 116 is not its floor.
- The bias fit does not use the same HG binning in every run (6 to 256 bins). The band shifted by under 1 HG ADC across 6, 40 and 256 bins in an earlier check, so it is compared here. The global sigma does move with binning and is not compared.

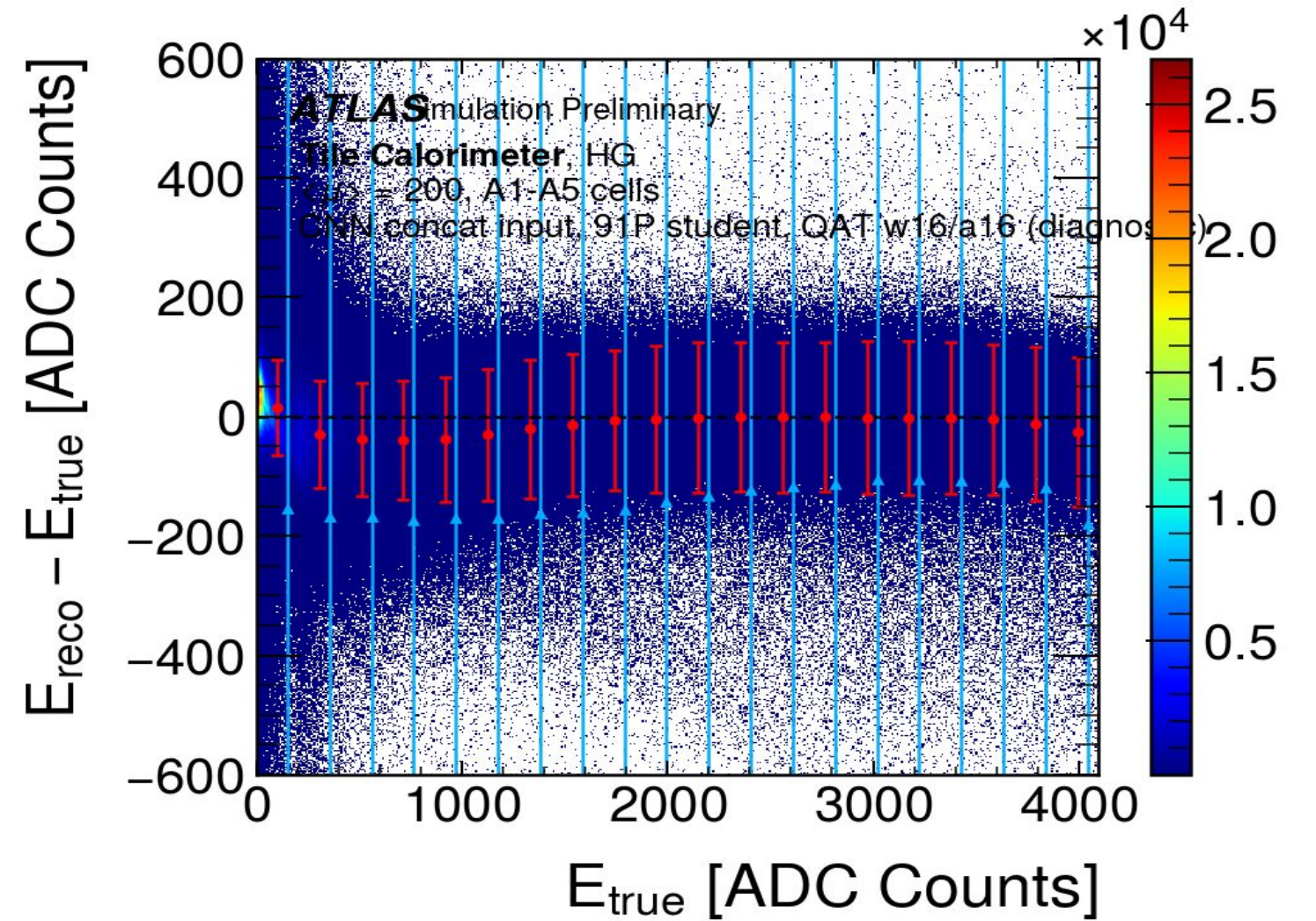
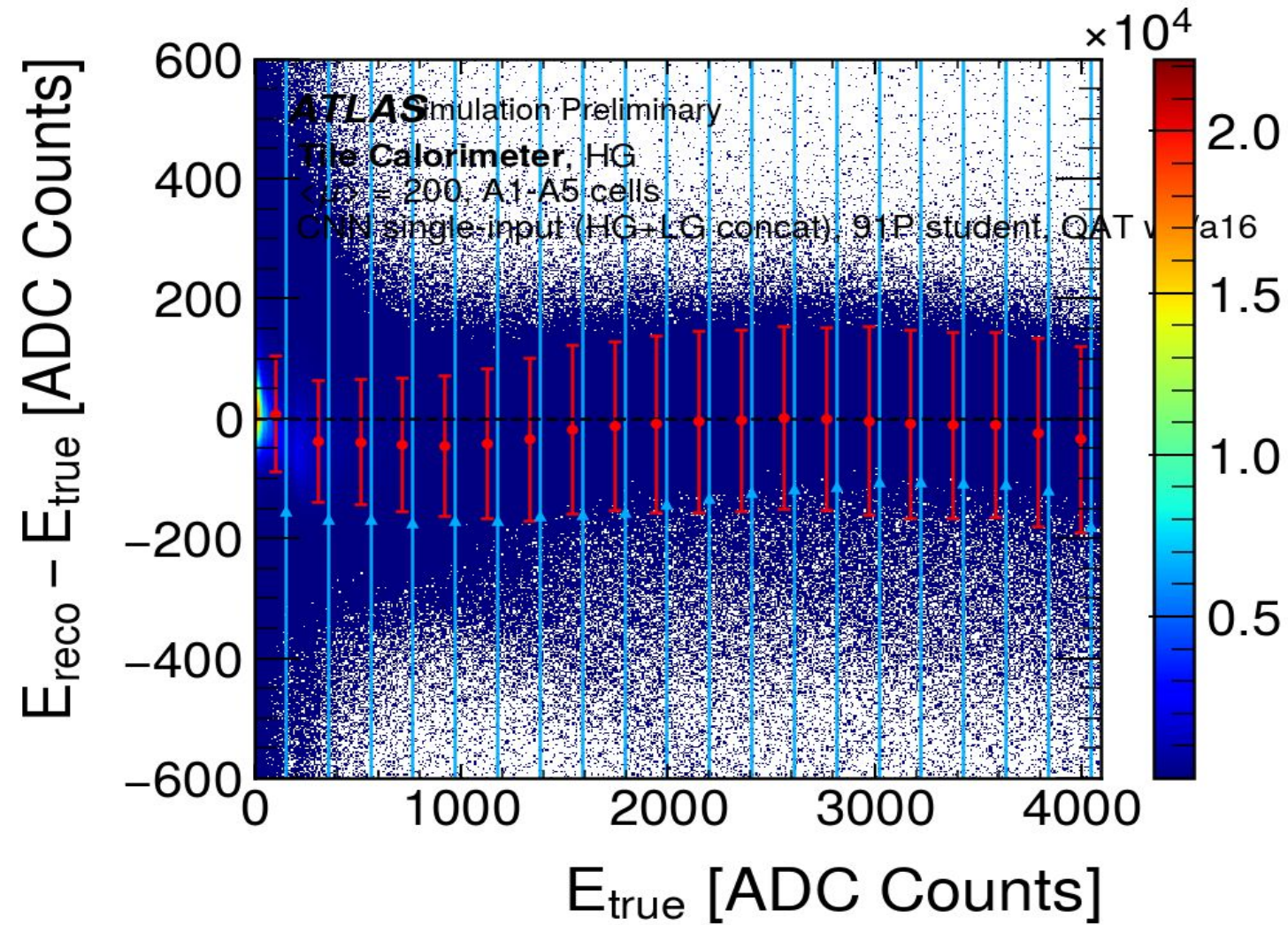
Below float: the cause is not established

The float student had converged, early-stopped with its best epoch at 85 of 115, so an undertrained baseline does not explain it. Two possibilities remain, and the existing runs cannot separate them.

1. The warm restart. Both quantized runs resume from the float checkpoint and get 100 more epochs at a fresh $1e-4$. That alone can find a better basin, and would have nothing to do with quantization.
 2. Quantization noise acting as a regularizer. A known effect, though at 16 bits the noise is around $1e-5$ relative, which makes it doubtful.
- A control run would settle it: continue the float model for the same 100 epochs at $1e-4$, with no quantizers. Landing near 116 would point to the restart, and the reference number would need updating. Staying near 127 would be more interesting.

No claim is made here that quantization improves resolution. If a confound has been missed, corrections are welcome.

The band, per energy bin



Red points are the per-bin mean and sigma after calibration. The mean is NOT flat, a low-energy dip survives, around -45 and -40 HG ADC respectively, with a further downturn at the top of the range. The two bias fits use different HG binning, so the dip depths are not directly comparable. What the extra weight bits shrink is the sigma bars, not the residual bias.

The export path, and what the C-simulation shows

- Brevitas -> export_qonnx -> QONNX cleanup -> hls4ml convert / compile / predict. It runs on the group cluster and needs only g++.
- predict() is not a float re-run of the model. It executes the generated HLS C++ with the same ap_fixed types, rounding and saturation that would be synthesised.
- It does show whether the fixed-point network keeps the trained resolution.
- It says nothing about DSPs, LUTs or latency. Those need build(), and Vivado.

Checking the firmware really is 8-bit weights, 16-bit activations

Every layer precision came back as "auto", meaning hls4ml inferred it from the QONNX Quant nodes rather than falling back to a default.

```
// firmware/defines.h (typedef names shortened)  
typedef ap_fixed<8,1,AP_RND_CONV,AP_SAT_SYM> weight_t; // conv / fc weights  
typedef ap_fixed<16,2> layer_t; // activations  
typedef ap_fixed<33,12> accum_t; // conv MAC result  
typedef ap_fixed<33,10> result_t; // network output
```

- The weights really are 8-bit in the datapath. Nothing was quietly widened.
- Rounding is AP_RND_CONV, round-to-nearest-even, which is what Brevitas RoundSte does.

LeakyReLU(0.25) comes out as a bit shift

The slope was picked as 2^{-2} , following a suggestion from an earlier meeting. This is what hls4ml generated:

```
// firmware/defines.h
typedef ap_ufixed<1,-1> param_t;    // one bit, weight 2^-2: exactly 0.25

// firmware/myproject.cpp
nnet::leaky_relu<...>(layer57_out, 0.25, layer44_out);

// nnet_utils/nnet_activation.h
res[i] = (datareg > 0) ? datareg : alpha * datareg;
```

- A direct compute with 0.25 as a compile-time literal, not a lookup table.
- Widening its output container changed the numbers by nothing.
- Zero DSP cost is expected, but that stays an expectation until synthesis says so.

A small residual, amplified by the gain

- The first C-simulation, at hls4ml's own inferred precisions, gave an HG band of 148.00 against Brevitas at 137.84. The LG bands agreed to within 0.3 LG ADC.
- The per-event residual was small, mean 1.20, max 4.84. In low gain.
- The target is ene_lo, and the high-gain view is ene_lo x 40.
- So 1.27 LG ADC becomes 50.8 HG ADC. In quadrature, $\sqrt{137.84^2 + 50.8^2} = 146.9$, against the 148.00 measured.
- This is the same x40 already seen on the HG bias, where a -0.5 LG ADC offset showed up as -20 HG ADC.

The residual was at first written off as negligible. It is not, once the gain is applied.

It turned out to be a container, not the quantization

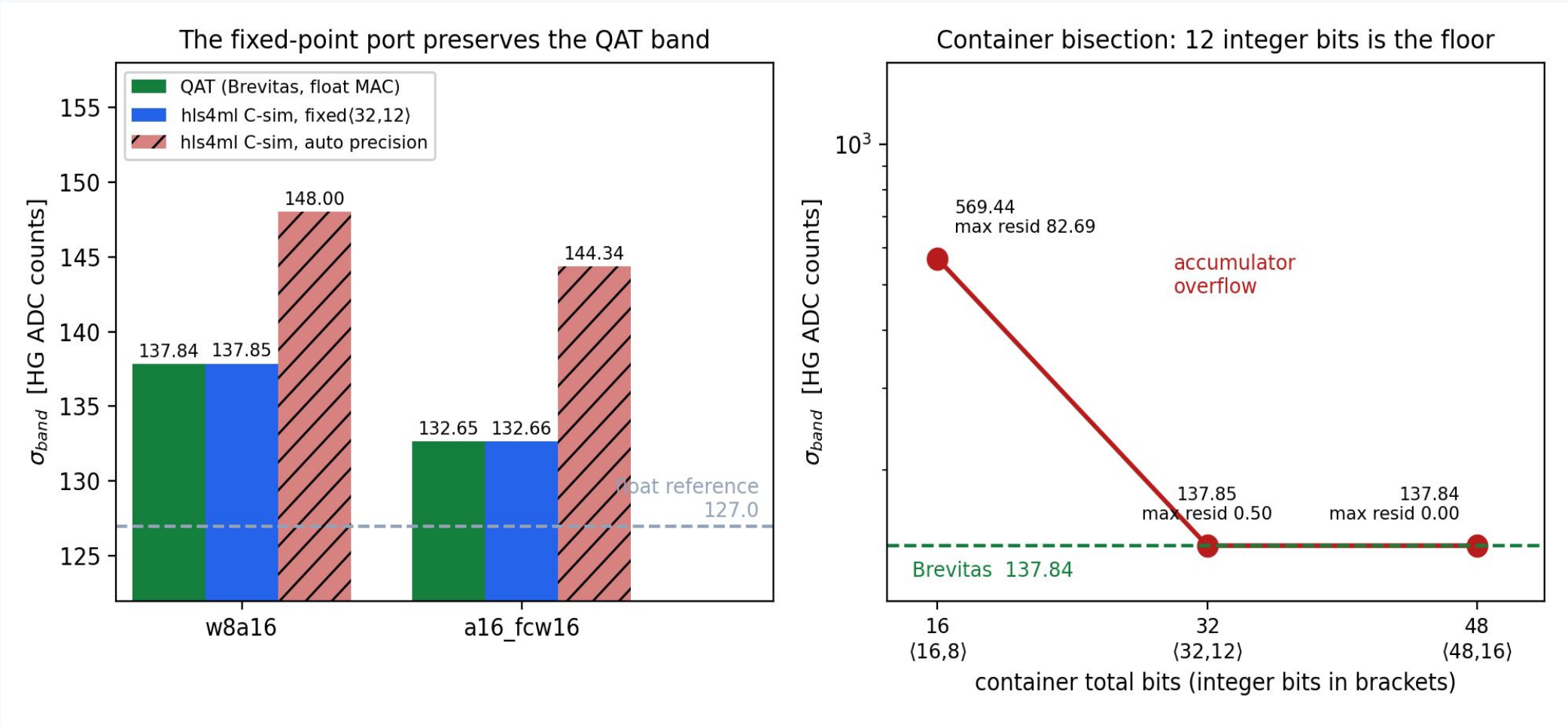
The accumulator, result and input container widths were bisected, leaving every Quant node untouched.

Residuals in LG ADC, band in HG ADC

Container	Residual mean / max	HG band	Verdict
fixed<48,16>	0.00 / 0.00	137.84	bit-exact
fixed<32,12>	0.03 / 0.50	137.85	the deployable choice
fixed<16,8>	12.57 / 82.69	569.44	overflows

- The dominant truncation was the input container. hls4ml sized it fixed<16,6>, which crushes the 24-bit S before it reaches the trained input quantizer.
- fixed<16,8> fails on range, not precision, 8 integer bits give +/-128, and the MAC sum needs around +/-2048. So 12 integer bits looks like a floor, with roughly 20 fractional bits enough.
- Sizing an accumulator to hold its own sum is ordinary fixed-point practice, and 48 bits is the native DSP48E2 accumulator width on this part. A second opinion on whether fixed<32,12> is the right place to stop would be welcome.

The fixed-point port reproduces the QAT band



Model	float	QAT	hls4ml
w8 / a16	127.0	137.84	137.85
+ fc 16b	127.0	132.65	132.66

HG band, ADC counts, calibrated

- Left: HG band for w8a16 and a16_fcw16 comparing QAT, hls4ml C-sim at fixed<32,12>, and hls4ml C-sim at auto precision. Right: container-width bisection.
- Both reproduce their QAT band to 0.01 HG ADC in the firmware datapath, so on the accuracy side the port looks faithful.
 - w16/a16 has not been exported yet. It is now the better model, so it should go through the same path.

What is blocked, and what needs a steer

Next steps

- The next step is `build(csim=False, synth=True)`, which needs Vivado. Access to a CERN machine. Could that be arranged?
- The weight result also changes what is worth synthesising, w8/a16 and w16/a16 side by side. A 16x16 multiply should still fit one DSP48E2, so the DSP count may not move while the LUT cost does. If that holds, w16/a16 may simply be the better choice, but it is not safe to assume so.
- There are no numbers at all for DSPs, LUT/FF or latency. Pre-synthesis csynth estimates are unreliable, so none are quoted here.

Questions better asked than guessed at

- Is the processing-latency budget 225 ns or 125 ns?
- Is xcku115-flva1517-2-e the right part, and is the flow Vivado HLS or Vitis?
- The output word is packed to 10-12 bits downstream. Should the last activation be trained at that width rather than 16

Summary

- Eight-bit activations do not resolve this energy range, and none of the four input-side transforms tried changed that.
- The weight sweep overturned an earlier conclusion. Eight-bit weights cost about 22 HG ADC of band, and w16/a16 reaches 116.0, below the float reference. One control run is owed before that is explained.
- The model exports cleanly through QONNX. The generated firmware carries genuinely 8-bit weights, and LeakyReLU(0.25) comes out as a bit shift.
- The bit-accurate C-simulation reproduces the QAT band to 0.01 HG ADC, once the input and accumulator containers are sized properly.
- Resources and latency are still unmeasured. Everything on the FPGA side stays an expectation until synthesis runs.

BACKUP

Backup: runs that would come next

1. Float control, 100 epochs at $1e-4$ from the float checkpoint. Separates the warm restart from quantization noise, and fixes the reference number. Cheap.
2. w12 / a16, the missing midpoint. A 12x16 multiply also fits one DSP48E2.
3. Continue w16 / a16 past the 100-epoch cap, since it had not converged.
4. Per-layer weight precision. The 28 fc weights at 16 bits bought 5.6 of the 22.3 HG ADC, so the other 63 conv weights hold the rest. Worth localising.
5. Export w16 / a16 through QONNX and C-sim, then synthesise it next to w8 / a16.

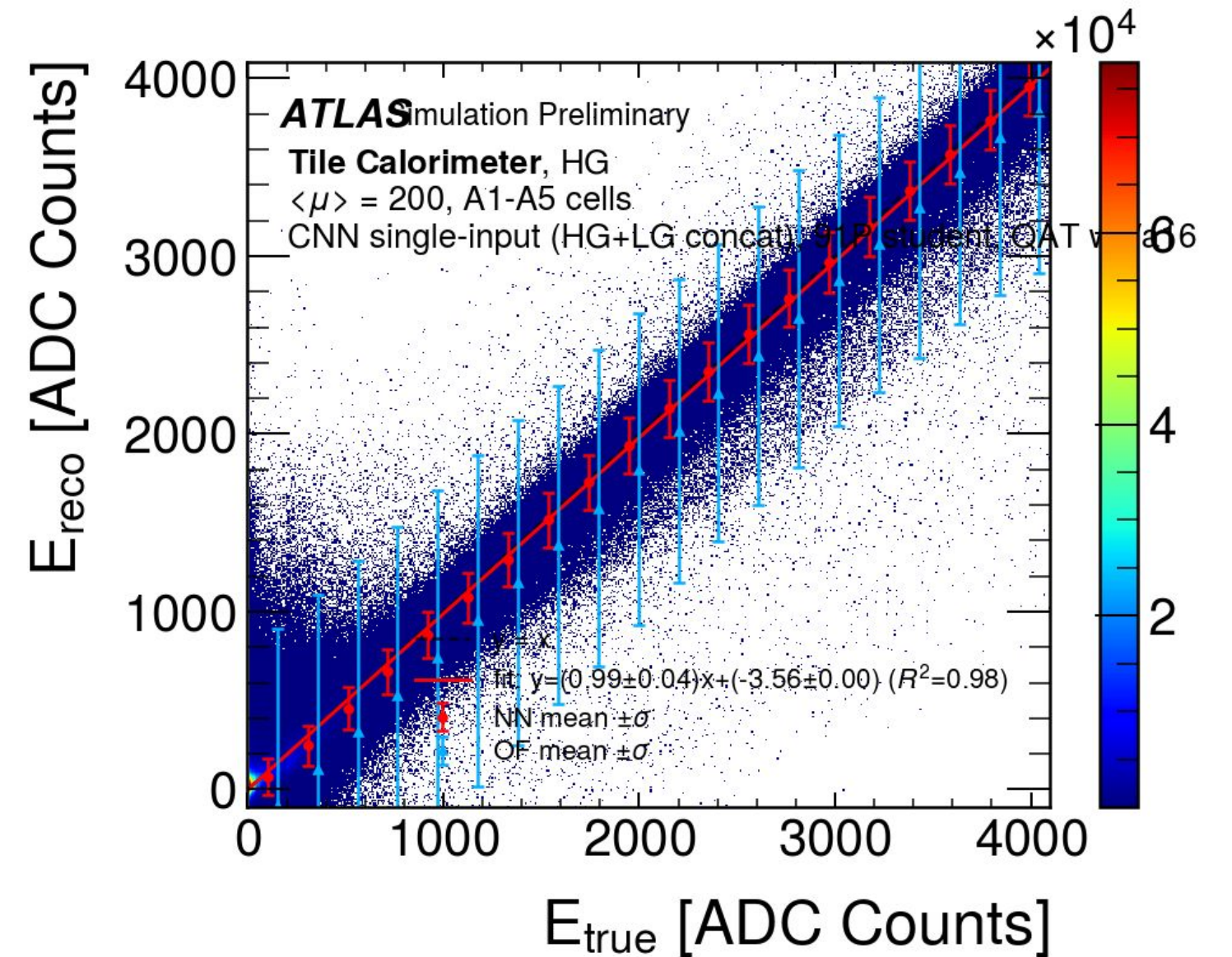
A steer on the order here, or on anything left off, would be welcome.

Backup: toolchain

- Working stack: brevitas 0.12.1, qonnx 1.0.0, hls4ml 1.3.0, onnx 1.22.0, onnxscript 0.7.0.
- hls4ml 1.0.0 pulls TensorFlow 2.14.1, which pins ml_dtypes 0.2.0, which breaks onnxscript. Since brevitas.export imports onnxscript, the export dies on import. hls4ml 1.3.0 keeps the QONNX frontend without TensorFlow.
- Conv padding was changed from "same" to an explicit pad. "same" exports as auto_pad=SAME_UPPER with no pads attribute, which the QONNX channels-last transform needs. It is bit-identical for odd kernels at stride 1.
- Cleaned graph: Quant x9, Transpose, Conv x2, LeakyRelu x2, Flatten, MatMul, Add. alpha=0.25 survives both the raw export and the cleanup.

Backup: the calibration, and what it does not fix

- The bias correction is a function of the reconstructed energy, not the truth. It is fitted on validation and applied to test. Bins are uniform, sigma is a plain standard deviation, nothing is trimmed or clipped.
- It does not flatten the mean. A low-energy dip of roughly -45 HG ADC survives calibration and does not move as the HG binning goes 6, 40, 256. That points to regression to the mean rather than under-correction, the bias lives in true energy, and the correction can only act on reco.
- The band is insensitive to that binning. The global sigma is not, because the near-zero pile-up bin dominates it. Hence the band is the quoted number.
- The HG binning is not yet uniform across the runs in this talk. Re-fitting all of them at 256 HG bins is a plotting pass, not a retrain.



E_{reco} against E_{true} for w8/a16,
after correction